P2: Os Algoritmos por Dentro

Este handout abre o código de `codigo/otimizacao_antes_depois.py`, seções 3 a 5. O objetivo é que você consiga explicar cada linha, dizer quanto tempo cada passo leva e por que ele está certo. Um físico computacional sempre pergunta: *como você sabe que o código faz o que você diz?* A resposta tem três partes: um algoritmo com prova (o pico de S_2), uma complexidade que você sabe estimar e testes contra força bruta.

Pré-requisitos: os handouts T1 (Exemplo 3) e T3 (pontes e betweenness). *Total: 17 pontos.*

1 O pipeline e o custo de cada passo

Para uma malha com N cruzamentos, E vias, R_d dias simulados e um orçamento de k vias:

Passo	Função	Custo
Betweenness com s origens	`carga_estrutural`	$O(s\,(E + N \log N))$, uma vez
Ruído por difusão (m passos)	`ruido_correlacionado`	$O(mE)$ por dia
Varredura: q_c, ligação crítica, R	`limiar_critico`	$O(E \log E)$
Red bonds e massas de corte	`red_bonds`	$O(N + E)$
Guloso com k passos	`guloso_ligacoes_criticas`	$O(k\,E \log E)$ por dia

Em Fortaleza ($E = 4166$), uma varredura leva 4,4 ms em Python puro. A simulação completa (40 dias, cinco estratégias, onze orçamentos, dois cenários) leva menos de um minuto.

2 Union-find

Queremos manter uma partição dos nós em clusters e fazer duas operações: *a qual cluster pertence o nó i?* e *junte os clusters de i e j*. Cada cluster é uma árvore, e a raiz é o "nome" do cluster.

```python
def raiz(i):
    while pai[i] != i:
        pai[i] = pai[pai[i]]          # compressão de caminho (halving)
        i = pai[i]
    return i
```

Ideia 1

Dois truques deixam o union-find praticamente $O(1)$ por operação.

- **União por tamanho:** pendure sempre a árvore menor na maior.
- **Compressão de caminho:** ao subir até a raiz, faça cada nó apontar para o avô (*halving*). Os próximos caminhos ficam mais curtos.

Com os dois, n operações custam $O(n\,\alpha(N))$, onde α é a inversa da função de Ackermann. Para qualquer N que caiba no universo, $\alpha(N) \le 4$.

[2] Problema 1. Sem compressão de caminho, só com união por tamanho, mostre que a profundidade de qualquer nó é no máximo $\log_2 N$. *Dica:* cada vez que a profundidade de um nó aumenta em 1, o que acontece com o tamanho do cluster dele?

3 A varredura de Newman–Ziff

Em vez de recalcular os componentes para cada q, ordenamos as vias da mais rápida para a mais lenta e as adicionamos uma a uma. Diminuir q é o mesmo que ir adicionando vias. Uma única passada percorre todos os q de 1 até 0.

```
melhor, critica = 0, -1
S1, robustez, q_anterior = 1, 0.0, 1.0
for e in np.argsort(-r).tolist():
    robustez += S1 * (q_anterior - r[e])   # degrau da escada S1(q)
    q_anterior = r[e]
    a, b = raiz(u[e]), raiz(v[e])
    if a == b:
        continue                          # ligação interna: fecha um ciclo
    if tam[a] < tam[b]:
        a, b = b, a                       # b é o cluster menor
    if tam[b] > melhor:
        melhor, critica = tam[b], e
    pai[b] = a                            # união por tamanho
    tam[a] += tam[b]
    S1 = max(S1, tam[a])
robustez += S1 * q_anterior               # de q = 0 até o menor r
```

Três coisas acontecem na mesma passada:

(a) **Pico de S_2 e ligação crítica.** Pelo Exemplo 3 do T1, $\max S_2 = \max_e \min(s_a, s_b)$. A linha `if tam[b] > melhor` guarda esse máximo e a ligação que o realiza. O r dela é q_c.

(b) **Robustez.** Entre dois valores consecutivos de r nada muda, então $S_1(q)$ é uma escada e $\int S_1 \, dq$ é uma soma de retângulos.

(c) **Ligações internas.** Se `a == b`, a via fecha um laço: não muda nenhum cluster. São exatamente as ligações "desperdiçadas" que fazem o p_c da rede quadrada ser maior que o da árvore (T1, Problema 4).

Exemplo 1. Seis nós e seis vias: $(1,2)$ com $r = 0{,}9$, $(3,4)$ com 0,8, $(5,6)$ com 0,7, $(2,3)$ com 0,6, $(4,5)$ com 0,5 e $(1,6)$ com 0,3. Rode o algoritmo à mão e encontre q_c, o pico de S_2 e R.

Solução.

via	r	clusters depois	$\min(s_a, s_b)$	melhor	S_1
$(1,2)$	0,9	{12}	1	1	2
$(3,4)$	0,8	{12},{34}	1	1	2
$(5,6)$	0,7	{12},{34},{56}	1	1	2
$(2,3)$	0,6	{1234},{56}	2	**2**	4
$(4,5)$	0,5	{123456}	2	2	6
$(1,6)$	0,3	— (interna)	—	2	6

O pico de S_2 é 2. O algoritmo guarda a *primeira* ligação que atinge o máximo, $(2,3)$, e devolve $q_c = 0{,}6$. Atenção: aqui há um **empate**. $S_2 = 2$ em todo o intervalo $q \in (0{,}5;\, 0{,}7]$, e quem de fato destrói o pico é $(4,5)$, em $q = 0{,}5$. Com empates, o pico é um platô, e o código escolhe uma ponta dele. Em Fortaleza o pico de S_2 tem centenas de nós, e empates exatos são raríssimos.

A escada de S_1: vale 1 em $(0{,}9;\, 1]$, 2 em $(0{,}6;\, 0{,}9]$, 4 em $(0{,}5;\, 0{,}6]$ e 6 em $[0;\, 0{,}5]$. Então

$$NR = 1(0{,}1) + 2(0{,}3) + 4(0{,}1) + 6(0{,}5) = 4{,}1, \qquad R = 4{,}1/6 = 0{,}683.$$

O código devolve exatamente (0,6; via (2, 3); 2; 0,6833). ◂

[2] Problema 2. No exemplo acima, troque a linha `if tam[b] > melhor` por `if tam[b] >= melhor`. Qual q_c o código devolve agora? Qual das duas convenções corresponde a "a ligação que destrói o pico"? Isso muda alguma conclusão física?

[2] Problema 3. Mostre que a soma do código é exatamente $\int_0^1 S_1(q)\,\mathrm{d}q$. Considere com cuidado os dois extremos: o intervalo $(r_{\max}, 1]$, onde nenhuma via está aberta, e o intervalo $[0, r_{\min}]$.

[2] Problema 4. Por que não uma grade em q? Uma alternativa seria calcular os componentes com `scipy` em 1000 valores de q. Compare o custo com o da varredura. E, mais importante, qual informação a grade *perde*? (O código usa a grade só para desenhar as curvas.)

4 Red bonds: pontes e a árvore de blobs

No ponto crítico, pegamos o cluster gigante (vias com $r \geq q_c$) e queremos todas as pontes, com suas massas de corte.

Ideia 2

(a) `nx.bridges` acha todas as pontes em $O(N + E)$ com o algoritmo de Tarjan: uma busca em profundidade que marca, para cada nó, o "ponto mais alto" que sua subárvore alcança por uma aresta de retorno (*low-link*). A aresta (pai, v) é ponte se a subárvore de v não consegue subir acima de v.

(b) Remova as pontes. Os componentes que sobram são os *blobs* (componentes 2-aresta-conexos).

(c) Contraia cada blob num nó com peso igual ao seu número de cruzamentos. As pontes viram as arestas de uma **árvore**.

(d) Numa árvore enraizada, a massa abaixo de cada aresta é a soma dos pesos da subárvore. Uma passada em pós-ordem (filhos antes dos pais) calcula todas: `massa_sub[pai] += massa_sub[filho]`.

(e) A massa de corte da ponte é min(subárvore, total − subárvore).

Custo total: $O(N+E)$. A alternativa ingênua (remover cada ponte e recontar os componentes) custa $O(B(N + E))$, com B pontes.

[3] Problema 5. Prove que o grafo obtido contraindo os blobs não tem ciclos. *Dica:* se houvesse um ciclo de blobs, o que seria verdade sobre as pontes desse ciclo?

[2] Problema 6. No teste do T1 (rede quadrada 40×40), o cluster crítico tinha em média 492 pontes, mas só 32 com massa de corte acima de 10% do cluster. Por que ordenar pela massa de corte é essencial para a estratégia "red bonds"? O que aconteceria se o código destravasse 41 pontes escolhidas ao acaso?

5 As estratégias

- **Ligações críticas (guloso iterativo):** encontre a ligação crítica, faça $r \to 1$, recalcule, repita k vezes. São $k + 1$ varreduras, e a curva $\Delta q_c(k)$ inteira sai de graça.

- **Red bonds (uma vez):** as k pontes de maior massa de corte no ponto crítico *original*, sem recalcular.
- **Mais lentas:** as k vias de menor r (força bruta).
- **Maior betweenness:** as k vias de maior b_e, que dependem só do mapa.
- **Aleatória:** o controle.

No *plano fixo*, as listas são montadas com os 20 primeiros dias e testadas nos 20 últimos. Para combinar as listas de vários dias usamos a **pontuação de Borda**: a via escolhida em j-ésimo lugar num dia ganha $1 - (j-1)/k$ pontos, e somamos sobre os dias de treino.

[2] Problema 7. Por que a pontuação de Borda e não simplesmente "quantos dias a via apareceu na lista"? Pense no que significa ser escolhida em primeiro lugar pelo guloso.

6 Validação e estatística

O arquivo `codigo/teste_algoritmos.py` compara a varredura com a força bruta (recalcular os componentes do zero em cada um dos E degraus de q, custo $O(E^2)$) em 30 grafos. Também testa que R nunca diminui ao destravar uma via (200 tentativas) e que toda red bond é de fato uma ponte, com massas em ordem decrescente.

As incertezas são erros padrão da média sobre os dias: $s/\sqrt{R_d}$.

[2] Problema 8. Em tempo real, em Fortaleza: ligações críticas $\Delta R = 0{,}0928 \pm 0{,}0036$; betweenness $\Delta R = 0{,}0526 \pm 0{,}0039$.

(a) Supondo as duas medidas independentes, a quantos desvios padrão está a diferença?

(b) Na verdade, as duas estratégias são avaliadas nos *mesmos* 40 dias. Por que o teste correto é o *pareado* (a média e o desvio de $\Delta R_{\text{crit}} - \Delta R_{\text{BC}}$ dia a dia)? O teste não pareado é conservador ou otimista aqui?

Soluções

Problema 1. A profundidade de um nó só aumenta quando a raiz dele é pendurada em outra, e pela união por tamanho isso só acontece quando o cluster dele é o *menor* dos dois. Então o novo cluster tem pelo menos o dobro do tamanho antigo. Depois de d aumentos, o cluster tem pelo menos 2^d nós, e $2^d \le N$ dá $d \le \log_2 N$.

Problema 2. Com >=, a última ligação com min = 2 ganha, que é $(4, 5)$, e $q_c = 0{,}5$. Essa é a ligação que destrói o pico, então >= corresponde literalmente à frase do T1. Fisicamente não muda nada: as duas convenções marcam as pontas do mesmo platô, e empates só acontecem quando o pico é pequeno. Num sistema grande, o pico tem centenas de nós e empates exatos são raríssimos.

Problema 3. Sejam $r_{(1)} \ge r_{(2)} \ge \cdots \ge r_{(E)}$ as velocidades ordenadas. Para $q \in (r_{(i+1)}, r_{(i)}]$ estão abertas exatamente as i primeiras vias, e $S_1 = S_1^{(i)}$. Em $(r_{(1)}, 1]$ nenhuma via está aberta e $S_1 = 1$: é o primeiro termo do laço, com `S1 = 1` e `q_anterior = 1`. Em $[0, r_{(E)}]$ todas estão abertas: é a linha depois do laço. Somando os retângulos, $\sum_i S_1^{(i)} \, (r_{(i)} - r_{(i+1)})$ com as duas pontas é exatamente a integral da escada.

Problema 4. Cada valor de q custa $O(N + E)$, então 1000 valores custam $\sim 1000(N + E)$, contra $O(E \log E)$ da varredura: umas 100 vezes mais caro em Fortaleza. Mais importante, a grade perde a *exatidão*. O pico de S_2 pode cair entre dois pontos da grade, e a grade não diz *qual* ligação causou a fusão. Sem a ligação crítica, o algoritmo guloso não existe.

Problema 5. Suponha um ciclo de blobs $B_1 - B_2 - \cdots - B_m - B_1$ ligados por pontes $e_1, \ldots, e_m$. Cada blob é conexo por dentro, então existe um ciclo no grafo original passando por e_1 (entra em B_2 por e_1, atravessa os blobs, volta a B_1 por e_m e fecha dentro de B_1). Mas uma aresta num ciclo não é ponte: removê-la deixa o resto do ciclo como caminho alternativo. Contradição. (Se o grafo original é conexo, o grafo de blobs é conexo e sem ciclos, ou seja, uma árvore.)

Problema 6. Quase todas as pontes são entradas de galhos pequenos (*dangling ends*). Destravá-las não muda a conectividade global, porque o galho continua pendurado do mesmo jeito. Só as pontes de massa grande separam pedaços macroscópicos. Destravar 41 pontes ao acaso daria um ganho próximo ao da estratégia aleatória, porque com grande probabilidade todas seriam pontes de galhos.

Problema 7. O guloso escolhe primeiro a ligação que segura o *maior* evento de fusão. As escolhas seguintes corrigem efeitos de segunda ordem, revelados só depois que a primeira foi destravada. A pontuação de Borda dá mais peso a quem é importante "de primeira" e evita que uma via que apareceu muitas vezes no fim da lista (ganho marginal pequeno) passe na frente de uma que apareceu poucas vezes, mas sempre no topo.

Problema 8. (a) $(0{,}0928 - 0{,}0526)/\sqrt{0{,}0036^2 + 0{,}0039^2} = 0{,}0402/0{,}0053 \approx 7{,}6\sigma$. (b) Num dia "fácil" ou "difícil" as duas estratégias tendem a ganhar mais ou menos juntas, então os erros são positivamente correlacionados. A variância da diferença é $\sigma_1^2 + \sigma_2^2 - 2\,\mathrm{Cov}$, *menor* que a soma. O teste não pareado superestima a incerteza da diferença: é conservador. Se a diferença já é significativa sem o pareamento, com ele é ainda mais.